Name: Hemant Marotkar
UID: 23015002 **Date: 1-10-26**

AIM:- Detect outliers/anomalies in network traffic using density-based clustering (DBSCAN)

[3]:
```
# Name: Hemant Marotkar ,UID: - 23015002

# Detect outliers/anomalies in network traffic using density-based clustering (DBSC

from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix, classification_report

# 1. Load the dataset
# The CSV file must be in the same folder as this Python file.
DATA_FILE = Path(r"D:\My Files\4th year\sem 7th\mlc lab\pract8\network_traffic_anom
df = pd.read_csv(DATA_FILE)

print("First 5 rows of dataset:")
print(df.head())

# 2. Select only numerical features for clustering
# DBSCAN is unsupervised, so we do not use the label column for training.
features = [
    "packet_size",
    "connection_duration",
    "src_bytes",
    "dst_bytes",
    "packet_rate",
    "error_rate"
]

X = df[features]

# 3. Standardize features
# Scaling is important because features such as src_bytes and error_rate
# have very different ranges.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Apply DBSCAN clustering
# eps: radius around a point to search for neighbors
# min_samples: minimum number of nearby points required to form a dense region
model = DBSCAN(eps=1.0, min_samples=3)
df["cluster"] = model.fit_predict(X_scaled)

# In DBSCAN, cluster = -1 means noise/outlier/anomaly.
df["predicted_anomaly"] = df["cluster"].apply(lambda value: "anomaly" if value == -

# 5. Display clustering result
print("\nDBSCAN clustering result:")
print(df[["packet_size", "connection_duration", "src_bytes", "dst_bytes", "packet_r

# 6. Compare with actual labels if labels are available
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(df["label"], df["predicted_anomaly"], labels=["normal", "ano

print("\nClassification Report:")
print(classification_report(df["label"], df["predicted_anomaly"]))

# 7. Reduce dimensions using PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

df["PCA1"] = X_pca[:, 0]
df["PCA2"] = X_pca[:, 1]

# 8. Plot normal points and anomalies
normal_points = df[df["predicted_anomaly"] == "normal"]
anomaly_points = df[df["predicted_anomaly"] == "anomaly"]

plt.figure(figsize=(8, 6))
plt.scatter(normal_points["PCA1"], normal_points["PCA2"], label="Normal")
plt.scatter(anomaly_points["PCA1"], anomaly_points["PCA2"], label="Anomaly", marker
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Network Traffic Anomaly Detection using DBSCAN")
plt.legend()
plt.grid(True)
plt.show()

# 9. Save output result to a new CSV file
OUTPUT_FILE = Path(r"D:\My Files\4th year\sem 7th\nlc lab\pract8\dbscan_anomaly_out
df.to_csv(OUTPUT_FILE, index=False)
print(f"\nOutput saved as {OUTPUT_FILE.name}")
```

First 5 rows of dataset:

	packet_size	connection_duration	src_bytes	dst_bytes	packet_rate	\
0	450	12	1200	3400	40	
1	520	15	1500	3600	42	
2	480	10	1100	3000	38	
3	610	18	1700	4200	45	
4	560	16	1600	3900	43	

	error_rate	label
0	0.01	normal
1	0.02	normal
2	0.01	normal
3	0.03	normal
4	0.02	normal

DBSCAN clustering result:

	packet_size	connection_duration	src_bytes	dst_bytes	packet_rate	\
0	450	12	1200	3400	40	
1	520	15	1500	3600	42	
2	480	10	1100	3000	38	
3	610	18	1700	4200	45	
4	560	16	1600	3900	43	
5	430	11	1000	3100	37	
6	590	17	1650	4000	44	
7	500	14	1400	3500	41	
8	470	13	1300	3300	39	
9	620	20	1800	4300	46	
10	300	50	500	20000	8	
11	320	55	600	21000	9	
12	310	48	550	19000	8	
13	290	52	520	20500	7	
14	305	49	580	19800	8	
15	250	80	400	30000	4	
16	260	75	450	28000	5	
17	255	78	420	29000	4	
18	270	76	470	28500	5	
19	240	82	390	31000	4	
20	900	3	9000	400	250	
21	1200	1	15000	100	350	
22	1700	8	26000	8000	90	
23	2000	1	50000	10	800	
24	700	100	200	50000	2	
25	50	5	100	50	600	

```
    error_rate  cluster predicted_anomaly
0         0.01        0             normal
1         0.02        0             normal
2         0.01        0             normal
3         0.03        0             normal
4         0.02        0             normal
5         0.01        0             normal
6         0.02        0             normal
7         0.02        0             normal
8         0.01        0             normal
9         0.03        0             normal
10        0.04        1             normal

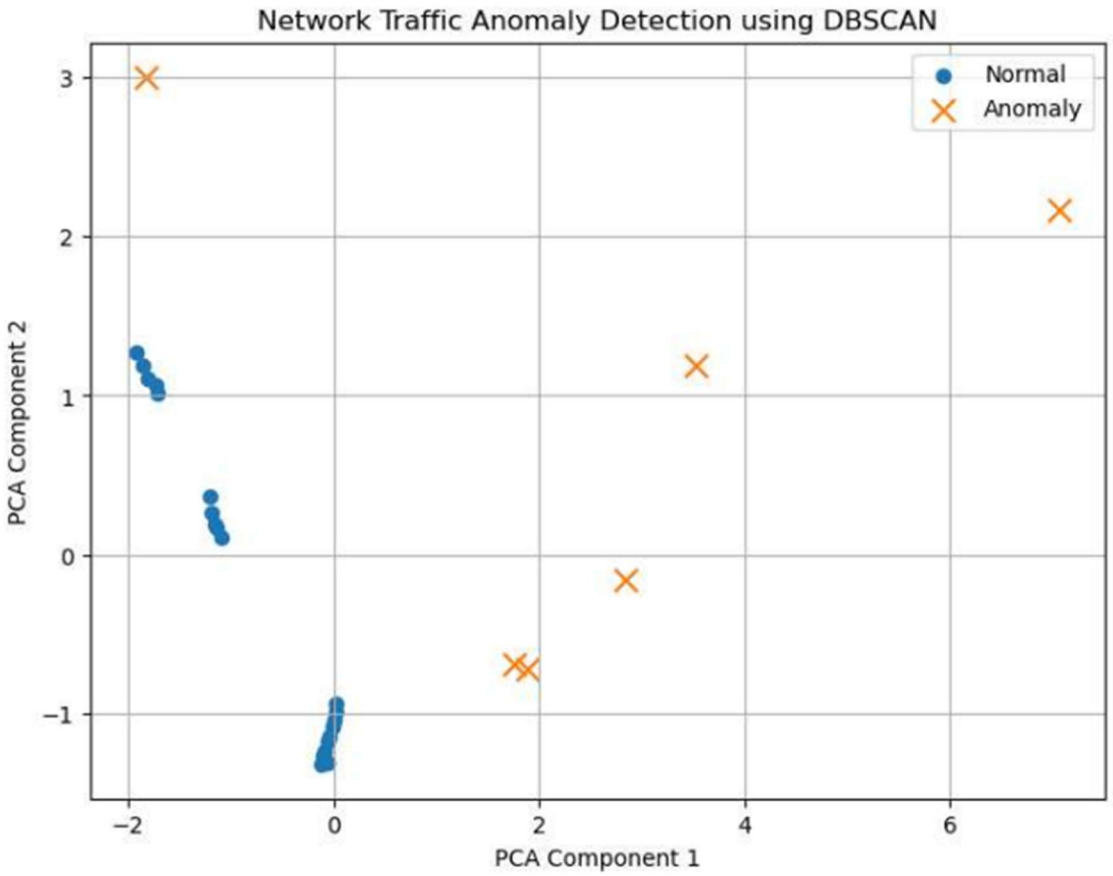

11        0.05        1             normal
12        0.04        1             normal
13        0.05        1             normal
14        0.04        1             normal
15        0.02        1             normal
16        0.03        1             normal
17        0.02        1             normal
18        0.03        1             normal
19        0.02        1             normal
20        0.25       -1            anomaly
21        0.40       -1            anomaly
22        0.70       -1            anomaly
23        0.90       -1            anomaly
24        0.20       -1            anomaly
25        0.55       -1            anomaly

Confusion Matrix:
[[20  0]
 [ 0  6]]

Classification Report:
              precision    recall  f1-score   support

     anomaly       1.00      1.00      1.00         6
      normal       1.00      1.00      1.00        20

    accuracy                           1.00        26
   macro avg       1.00      1.00      1.00        26
weighted avg       1.00      1.00      1.00        26
```




```
Output saved as dbscan_anomaly_output.csv
```

Conclusion:

We performed an experiment to detect outliers/anomalies in network traffic using density-based clustering (DBSCAN)